



INSTITUCION:

INSTITUTO SUPERIOR TECNOLÓGICO DE TURISMO Y PATRIMONIO YAVIRAC

CARRERA:

DESARROLLO DE SOFTWARE

ASIGNATURA:

DEVOPS

DOCENTE:

MSC. BYRON MORENO

ESTUDIANTES:

FRANCISCO ESTEBAN RAMIREZ AZA

DOMENICA DAYANA ESPINOSA IMBAQUINGO

CURSO:

QUINTO "A"

TEMA:

TRABAJO COLABORATIVO EN PAREJAS

1. INVESTIGACIÓN

Control de versiones

Definición: Es un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo, permitiendo recuperar versiones específicas en el futuro.

Utilidad: Permite a los equipos de desarrollo trabajar simultáneamente en el mismo código sin sobrescribir el trabajo de otros, manteniendo un historial completo para revertir errores si es necesario.

Ejemplo práctico: Un equipo trabaja en una aplicación de entregas; si una nueva actualización rompe el sistema de pagos, el control de versiones permite regresar el código al estado exacto en el que funcionaba el día anterior en cuestión de segundos.

Fuente bibliográfica: Sommerville, I. (2016). Ingeniería de Software (10.^a ed.). Pearson Educación.

Git

Definición: Es un sistema de control de versiones de código abierto, gratuito y de tipo distribuido, creado por Linus Torvalds en 2005.

Utilidad: Al ser distribuido, cada desarrollador tiene una copia completa del historial de código en su máquina. Es extremadamente rápido, eficiente con proyectos grandes y gestiona ramas (branches) de forma ligera.

Ejemplo práctico: Un programador usa los comandos git init en su terminal para empezar a registrar los cambios de una nueva página web, y luego usa git status para ver qué archivos ha modificado.

Fuente bibliográfica: Chacon, S., & Straub, B. (2014). Pro Git (2.^a ed.). Apress.

<https://git-scm.com/book/es/v2>

GitHub

Definición: Es una plataforma de alojamiento en la nube para proyectos que utilizan el sistema de control de versiones Git.

Utilidad: Facilita el trabajo colaborativo proporcionando una interfaz gráfica web, herramientas de gestión de proyectos, control de acceso, revisión de código y automatización de flujos de trabajo.

Ejemplo práctico: Los dos integrantes de este grupo suben el código de su proyecto de clase a GitHub para que ambos puedan descargar los avances del otro y el profesor pueda calificar la entrega.

Fuente bibliográfica: Loeliger, J., & McCullough, M. (2012). Version Control with Git. O'Reilly Media.

Repositorio local

Definición: Es la copia del proyecto que reside físicamente en el disco duro de la computadora del desarrollador, incluyendo su propio historial de Git.

Utilidad: Permite al desarrollador trabajar sin conexión a internet, realizar pruebas, hacer modificaciones y guardar historiales de cambios de forma privada antes de compartirlos.

Ejemplo práctico: El Estudiante A escribe código en su computadora portátil dentro de la carpeta C:/Proyectos/MiApp. Todos los archivos guardados allí forman parte de su repositorio local.

Fuente bibliográfica: Bell, P., & Beer, B. (2014). Introducing GitHub: A Non-Technical Guide. O'Reilly Media.

Repositorio remoto

Definición: Es la versión del proyecto que se encuentra alojada en un servidor en internet o en una red compartida (como GitHub), accesible por todo el equipo.

Utilidad: Funciona como la "fuente única de verdad" y respaldo centralizado donde los desarrolladores envían sus cambios locales para que otros los descarguen.

Ejemplo práctico: La carpeta del proyecto subida en la dirección web <https://github.com/usuario/proyecto-colaborativo> donde ambos estudiantes suben sus aportes.

Fuente bibliográfica: Chacon, S., & Straub, B. (2014). Pro Git (2.^a ed.). Apress.

Commit

Definición: Es una instantánea o "foto" del estado del código en un momento específico que se guarda permanentemente en el historial de Git.

Utilidad: Funciona como un punto de control detallado. Cada commit tiene un identificador único (SHA-1) y un mensaje explicativo para entender qué cambió y por qué.

Ejemplo práctico: Tras diseñar el formulario de inicio de sesión, el desarrollador ejecuta el comando `git commit -m "Añadir diseño visual y validación del formulario de login"`.

Fuente bibliográfica: Ponuthorai, S., & Loeliger, J. (2022). Version Control with Git (3.^a ed.). O'Reilly Media.

Branch (rama)

Definición: Es una línea de desarrollo independiente que se bifurca de la línea principal (usualmente llamada main o master).

Utilidad: Permite a un desarrollador crear un entorno aislado para experimentar o construir una nueva función sin alterar el código que ya está funcionando en producción.

Ejemplo práctico: El Estudiante B crea una rama llamada feature-contacto para programar una sección de contacto en la web, asegurándose de que si el código falla, la página web principal (main) siga operando con normalidad.

Fuente bibliográfica: Prettyman, S. (2016). Learn Git in a Month of Lunches. Manning Publications.

Merge

Definición: Es la operación que une o fusiona dos o más ramas de desarrollo, integrando los cambios de una rama secundaria dentro de una rama objetivo (como main).

Utilidad: Permite unificar el trabajo disperso de varios desarrolladores o integrar nuevas características terminadas de vuelta al tronco principal del proyecto.

Ejemplo práctico: Una vez que la sección de contacto en la rama feature-contacto funciona perfectamente y fue revisada, se ejecuta un merge para que esos cambios se sumen a la rama principal main.

Fuente bibliográfica: Chacon, S., & Straub, B. (2014). Pro Git (2.^a ed.). Apress.

Pull Request (PR)

Definición: Es una solicitud formal que hace un desarrollador en plataformas como GitHub para avisar al equipo que ha terminado una tarea y desea fusionar sus cambios en la rama principal.

Utilidad: Abre un espacio de discusión visual donde otros programadores revisan el código expuesto, hacen comentarios línea por línea, sugieren mejoras y aprueban o rechazan los cambios antes del merge.

Ejemplo práctico: El Estudiante B termina su parte de la práctica, sube su rama a GitHub y presiona el botón "New Pull Request" para que el Estudiante A valide su código.

Fuente bibliográfica: Bell, P., & Beer, B. (2014). Introducing GitHub: A Non-Technical Guide. O'Reilly Media.

Fork

Definición: Es una copia exacta de un repositorio ajeno que se guarda bajo la cuenta de un usuario propio en GitHub.

Utilidad: Permite proponer cambios a proyectos de código abierto de los cuales no eres colaborador directo, o tomar un proyecto existente como base para crear uno nuevo de forma independiente.

Ejemplo práctico: Un programador encuentra una librería útil en GitHub pero quiere adaptarla a sus necesidades. Hace un Fork del repositorio original para modificarlo en su propio perfil sin alterar el proyecto original.

Fuente bibliográfica: Gousios, G., Pinzger, M., & Deursen, A. V. (2014). An exploratory study of the pull-based software development model. Proceedings of the 36th International Conference on Software Engineering, 345-355.

Conflictos de integración

Definición: Es una situación anómala en la que Git no puede fusionar automáticamente dos ramas debido a que se han modificado las mismas líneas de un archivo con contenidos diferentes en ambas ramas.

Utilidad: Detiene el proceso de fusión de forma segura para evitar que el sistema sobrescriba código destructivamente de manera automatizada, obligando a los desarrolladores a decidir qué código conservar.

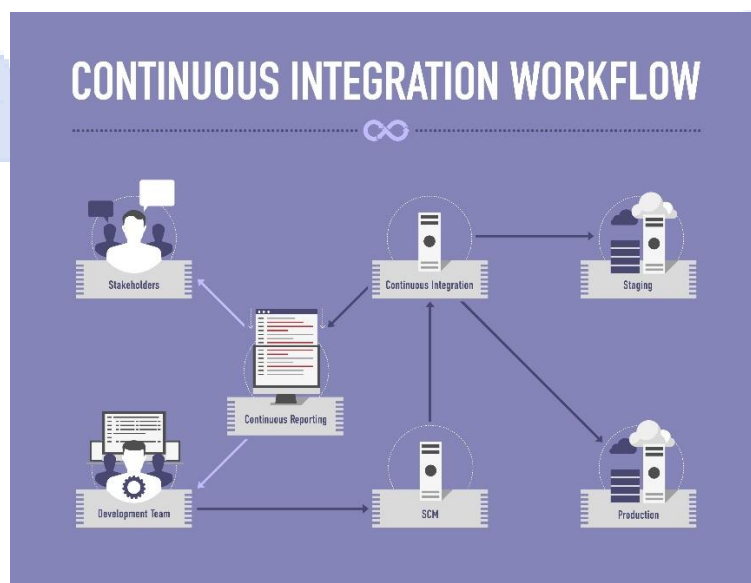
Ejemplo práctico: El Estudiante A cambia el color del botón a "Azul" en main. El Estudiante B cambia el mismo botón a "Verde" en su rama. Al intentar hacer el merge, Git se detiene e indica que los estudiantes deben resolver la discrepancia manualmente.

Fuente bibliográfica: Bird, C., Bird, L., & Zimmermann, T. (2015). The Art and Science of Analyzing Software Data. Morgan Kaufmann.

Integración Continua (CI)

Definición: Es una práctica de desarrollo de software donde los miembros del equipo integran sus cambios frecuentemente en el repositorio principal, activando pruebas automáticas cada vez que lo hacen.

Utilidad: Detecta errores y fallos de código de forma temprana en el ciclo de desarrollo, evitando sorpresas desagradables al final del proyecto.



Fuente: Shutterstock

Ejemplo práctico: Cada vez que un estudiante sube un commit a GitHub, un sistema automatizado ejecuta pruebas unitarias para validar que el formato del código sea correcto y que no rompa funciones existentes.

Fuente bibliográfica: Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.

Entrega Continua (Continuous Delivery)

Definición: Es la evolución de la Integración Continua, donde el código que pasa las pruebas automáticas se empaqueta y se prepara automáticamente en un entorno listo para ser desplegado.

Utilidad: Garantiza que el código esté siempre en un estado "liberable", reduciendo el riesgo de los lanzamientos y permitiendo que desplegar el software dependa únicamente de pulsar un botón por decisión humana.

Ejemplo práctico: El sistema compila automáticamente la aplicación y genera un archivo instalador .apk listo para producción, guardándolo en un servidor a la espera de que el gerente apruebe su salida al mercado.

Fuente bibliográfica: Chen, L. (2015). Continuous delivery: Overcoming adoption challenges. IEEE Software, 32(2), 50-54.

Despliegue Continuo (Continuous Deployment)

Definición: Es una práctica avanzada donde cada cambio que pasa con éxito todas las etapas de la canalización de pruebas automatizadas se publica directamente en el entorno de producción (clientes reales) sin intervención humana.

Utilidad: Acelera drásticamente el tiempo de llegada al mercado y elimina los cuellos de botella manuales o burocráticos en la entrega de valor.

Ejemplo práctico: Un programador corrige una falta de ortografía en la interfaz de usuario, hace commit y push; tras 5 minutos de pruebas automáticas exitosas, el cambio ya está visible en la página web oficial para todos los usuarios del mundo.

Fuente bibliográfica: Humble, J., & Farley, D. (2010). Continuous Delivery. Addison-Wesley Professional.

GitHub Actions

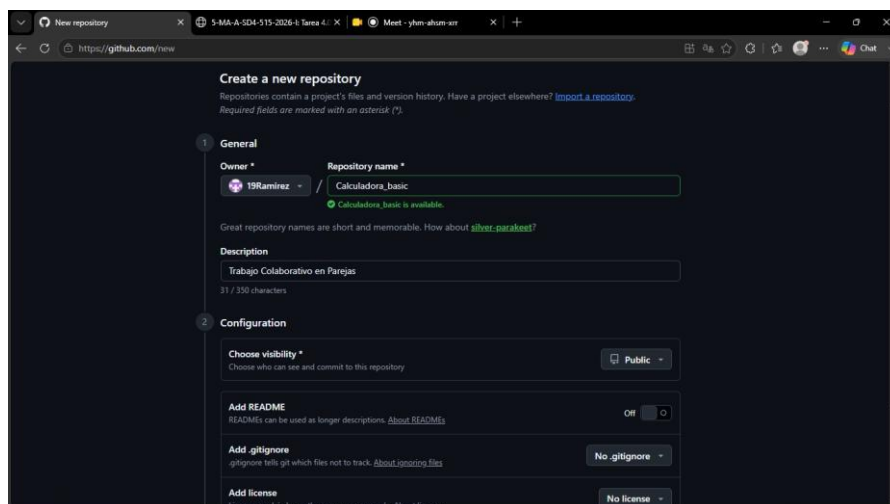
Definición: Es la herramienta nativa de automatización de flujos de trabajo e integración/despliegue continuo (CI/CD) integrada directamente dentro de GitHub.

Utilidad: Permite crear workflows personalizados mediante archivos de configuración en formato YAML para compilar, probar, empaquetar o desplegar proyectos basados en eventos del repositorio (como un Push o un Pull Request).

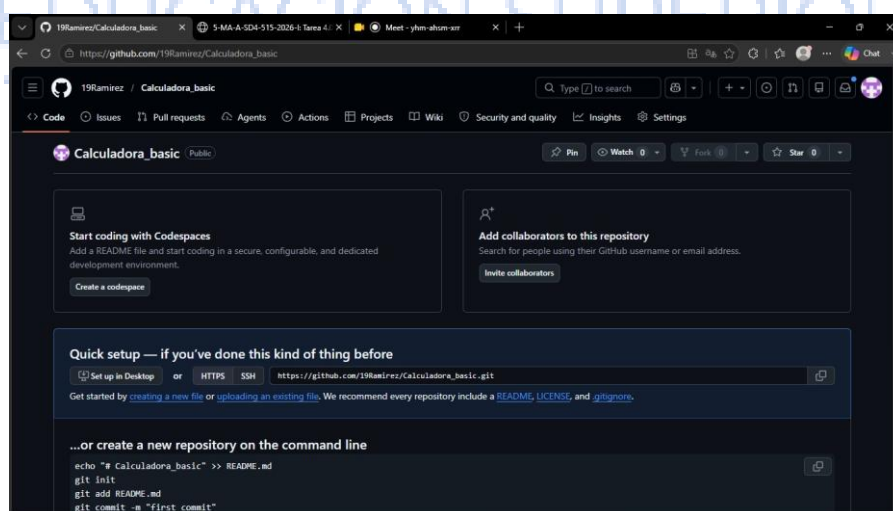
Ejemplo práctico: Configurar un archivo `.github/workflows/main.yml` en la práctica para que revise automáticamente si el código HTML de los estudiantes cumple con los estándares web antes de permitir el Merge.

Fuente bibliográfica: Callanan, J. (2021). Automating workflows with GitHub Actions. Packt Publishing.

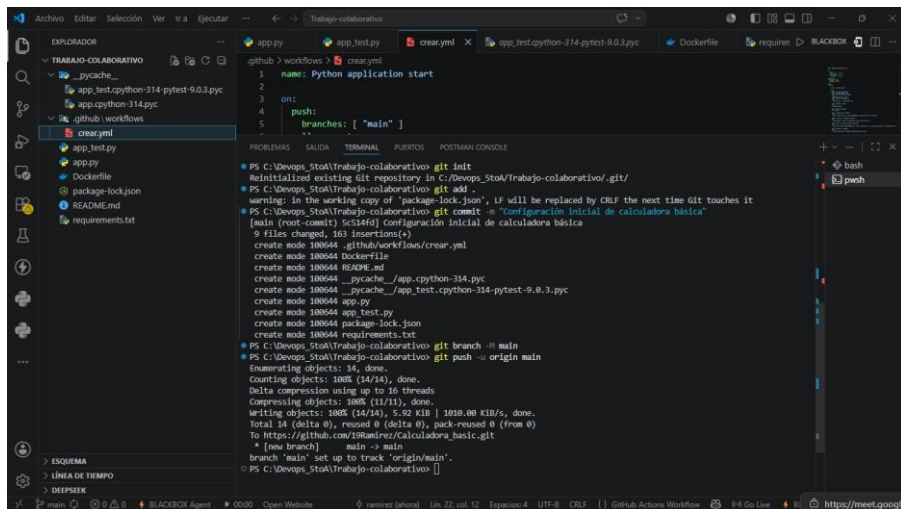
2. PRACTICO



Creación del repositorio en GitHub



Repositorio de GitHub creado



```
PS C:\Devops_5toA\Trabajo-colaborativo> git init
Reinitialized existing Git repository in C:\Devops_5toA\Trabajo-colaborativo/.git/
PS C:\Devops_5toA\Trabajo-colaborativo> git add .
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
PS C:\Devops_5toA\Trabajo-colaborativo> git commit -m "Configuración inicial de calculadora básica"
[main (root-commit) 5c514fd] Configuración inicial de calculadora básica
 9 files changed, 163 insertions(+)
 create mode 100644 .github/workflows/creayml
 create mode 100644 Dockerfile
 create mode 100644 README.md
 create mode 100644 __pycache__/_app.cpython-314.pyc
 create mode 100644 __pycache__/_app_test.cpython-314-pytest-9.0.3.pyc
 create mode 100644 app.py
 create mode 100644 app_test.py
 create mode 100644 package-lock.json
 create mode 100644 requirements.txt
PS C:\Devops_5toA\Trabajo-colaborativo> git branch -M main
PS C:\Devops_5toA\Trabajo-colaborativo> git push -u origin main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 16 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (14/14), 5.92 KiB | 1010.00 KiB/s, done.
Total 14 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/19Ramirez/Calculadora_basica.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Devops_5toA\Trabajo-colaborativo>
```

Git Log como commit inicial

```
C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo>git init
Reinitialized existing Git repository in C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo/.git/

C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo>git clone https://github.com/19Ramirez/Calculadora_basica.git
Cloning into 'calculadora_basica'...
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (14/14), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 14 (delta 0), reused 14 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (14/14), 5.92 KiB | 758.00 KiB/s, done.
```

Clonar el repositorio

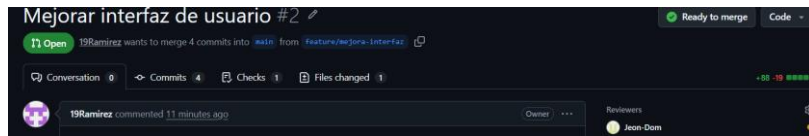
```
PS C:\Devops_5toA\Trabajo-colaborativo> git checkout -b feature/mejora-interfaz
Switched to a new branch 'feature/mejora-interfaz'
PS C:\Devops_5toA\Trabajo-colaborativo> git branch -a
* feature/mejora-interfaz
main
remotes/origin/main
```

```
Microsoft Windows [Versión 10.0.26200.8524]
(c) Microsoft Corporation. Todos los derechos reservados.

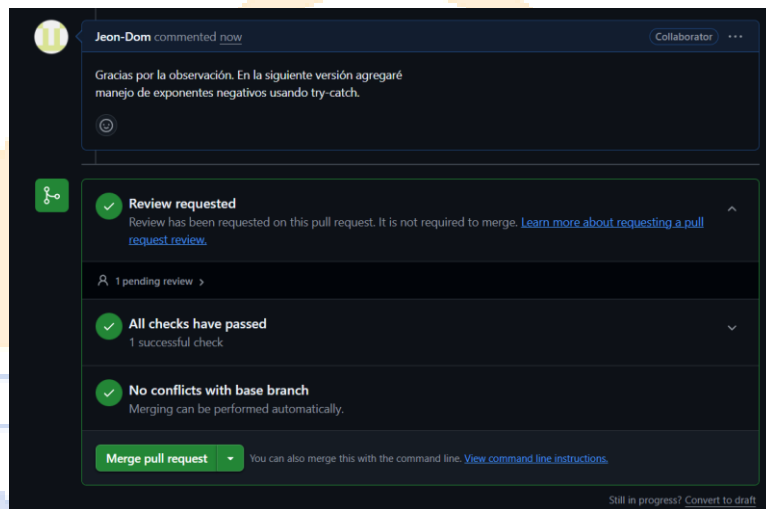
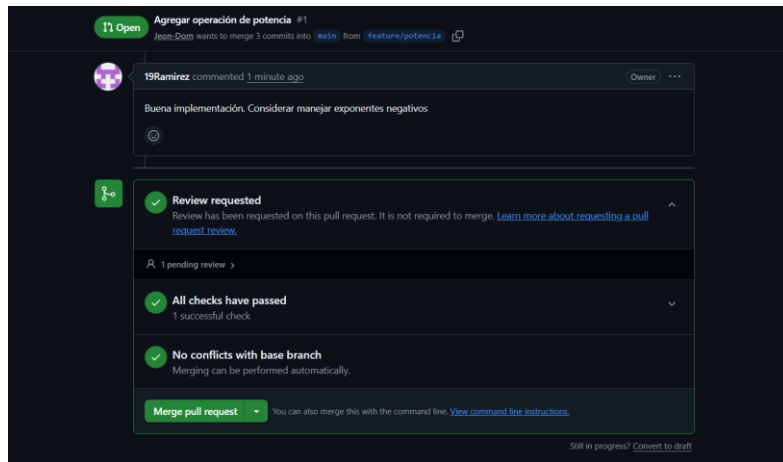
C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo\Calculadora_basica>git checkout -b feature/potencia
Switched to a new branch 'feature/potencia'

C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo\Calculadora_basica>git branch
* feature/potencia
main
```

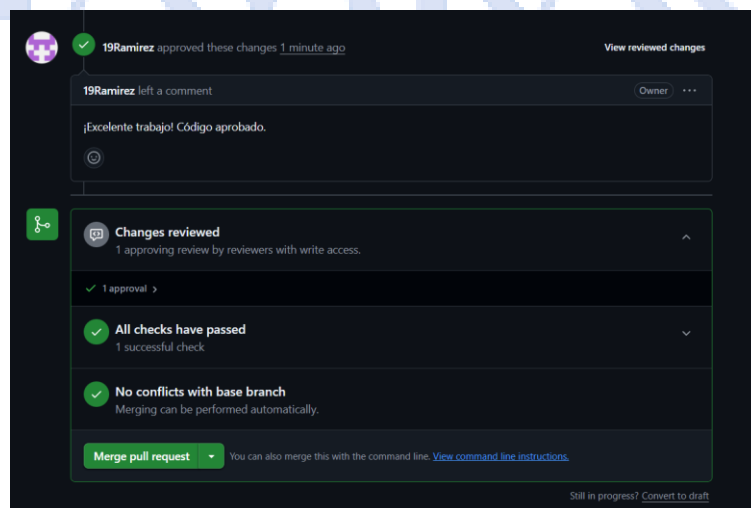
Creación de ramas



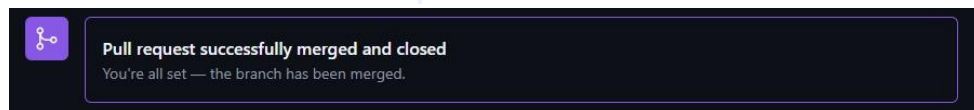
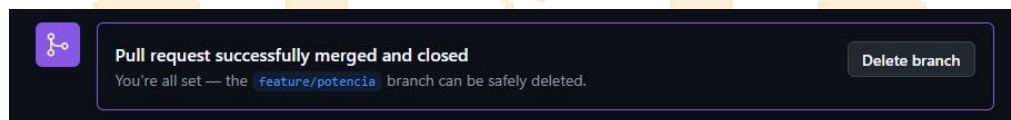
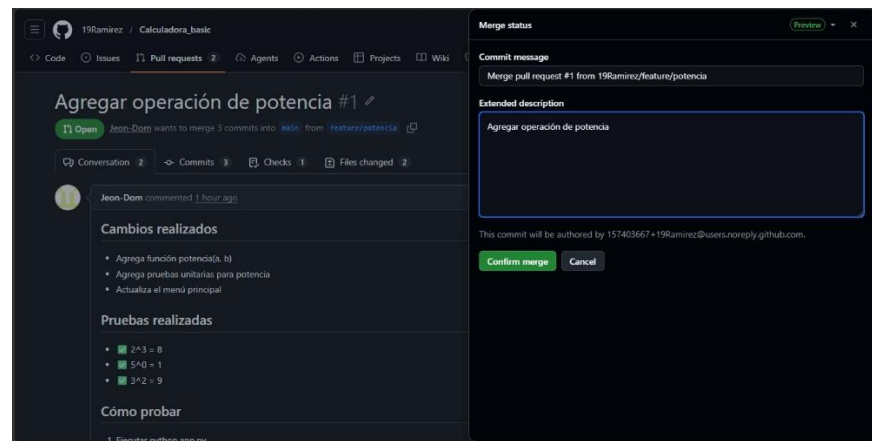
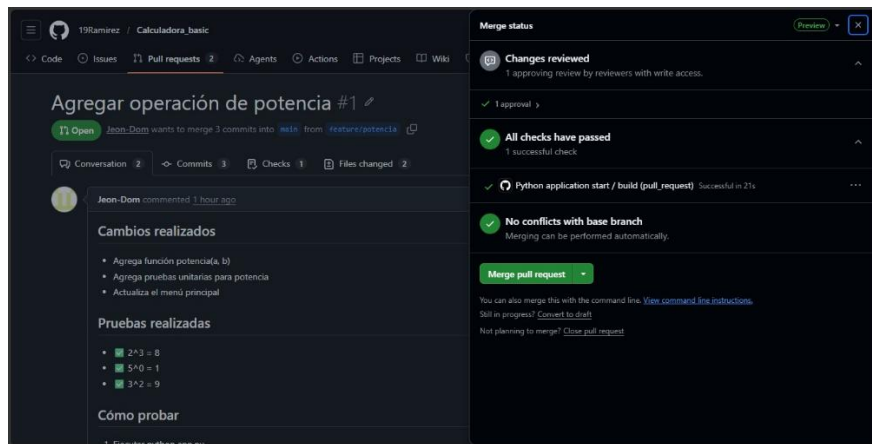
Pull Request creado por cada colaborador



Revisión de código de cada colaborador



Aprobación de código



Ejecución del proceso de Merge.

```
PS C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo\Calculadora_basic> git merge feature/potencia
fatal: You have not concluded your merge (MERGE_HEAD exists).
Please, commit your changes before you merge.
```

```
def calculadora_interactiva():
    """Versión interactiva para uso normal"""
    print("== MI SUPER CALCULADORA POTENTE v2.0 ==")
    print("1. Suma")
    print("2. Resta")
    print("3. Multiplicación")
    print("4. División")
    print("5. Potencia")
    print("6. Salir")
```

```

PS C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo\Calculadora_basico> git status
On branch main
Your branch is up to date with 'origin/main'.

All conflicts fixed but you are still merging.
(use "git commit" to conclude merge)

Changes to be committed:
  modified:   app.py

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   .github/workflows/create.yml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    t.py

PS C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo\Calculadora_basico> git add app.py
PS C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo\Calculadora_basico> git commit -m "merge: Resolver conflicto de título entre main y feature/potencia"
[main bbea48c] merge: Resolver conflicto de título entre main y feature/potencia
PS C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo\Calculadora_basico> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 376 bytes | 376.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/19Ramirez/Calculadora_basico.git
22e37c8..bbea48c  main -> main

```

Generación y resolución de conflictos de integración, se combinó títulos para la resolución del conflicto.

All workflows					Filter workflow runs
Showing runs from all workflows					
5 workflow runs					Workflow ▾ Event ▾ Status ▾ Branch ▾ Actor ▾
✓	merge: Resolver conflicto de título entre main y feature/potencia	main	4 minutes ago	26s	...
Python application start #10: Commit bbea48c pushed by Jeon-Dom					
✓	Merge pull request #1 from 19Ramirez/feature/potencia	main	27 minutes ago	28s	...
Python application start #9: Commit 22e37c8 pushed by 19Ramirez					
✓	Mejorar interfaz de usuario	feature/mejora-interfaz	49 minutes ago	25s	...
Python application start #6: Pull request #2 synchronize by 19Ramirez					
✓	Agregar operación de potencia	feature/potencia	1 hour ago	25s	...
Python application start #4: Pull request #1 synchronize by Jeon-Dom					
✗	Configuración inicial de calculadora básica	main	Today at 11:29 AM	13s	...
Python application start #1: Commit 5c514fd pushed by 19Ramirez					

Ejecución exitosa del workflow de GitHub Actions.

```

PS C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo\Calculadora_basico> git log --oneline --graph --all
* bbea48c (HEAD -> main, origin/main, origin/HEAD) merge: Resolver conflicto de título entre main y feature/potencia
|
| * efe19bb (origin/feature/potencia, feature/potencia) style: Cambiar título a versión potente
| * 22e37c8 Merge pull request #1 from 19Ramirez/feature/potencia
|/
| * 100cb10 fix: Corregir autenticación
| * edcd976 feat: Agrega operación de potencia con sus pruebas unitarias
| * 85567ab Agrega operación de potencia con sus pruebas unitarias
| * a90b596 Update career and parallel description in README
| * c41d05b Update README.md
|/
* 5c514fd Configuración inicial de calculadora básica
PS C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo\Calculadora_basico> git shortlog -sn
5 Jeon-Dom
3 19Ramirez
1 ramirez
PS C:\Users\USUARIO\Documents\Trabajos DevOps\Trabajo colaborativo\Calculadora_basico> git branch -a
feature/potencia
* main
remotes/origin/HEAD -> origin/main
remotes/origin/feature/potencia
remotes/origin/main

```

Estado final del repositorio después de integrar todos los cambios.

3. URL del repositorio GitHub

https://github.com/19Ramirez/Calculadora_basic.git

4. Conclusiones individuales de cada integrante.

Integrante 1: Mejora de Interfaz

Durante este proyecto, pude implementar mejoras significativas en la interfaz de usuario de la calculadora, incorporando elementos visuales como emojis y caracteres especiales que hacen la experiencia más amigable e intuitiva. Aprendí que el diseño de interfaz no solo es cuestión de estética, sino también de usabilidad, ya que los símbolos visuales ayudan a los usuarios a identificar rápidamente cada operación. Enfrenté el desafío de mantener la funcionalidad mientras mejoraba la presentación, lo que me enseñó a equilibrar forma y función en el desarrollo de software. También aprendí a manejar adecuadamente los errores de entrada en entornos automatizados como GitHub Actions, adaptando el código para que funcione tanto en modo interactivo como en modo prueba. Esta experiencia reforzó mi comprensión sobre la importancia de escribir código robusto que pueda ejecutarse en diferentes entornos sin fallar.

Integrante 2: Funcionalidades Matemáticas

Durante el desarrollo de este proyecto, fui responsable de implementar las operaciones matemáticas fundamentales de la calculadora, incluyendo suma, resta, multiplicación y división. Aprendí que la precisión en los cálculos numéricos es crucial, especialmente al manejar números decimales y divisiones que pueden resultar en números periódicos, por lo que decidí usar el tipo float de Python para permitir una mayor versatilidad en los cálculos. Enfrenté el desafío específico de manejar la división entre cero, implementando una validación que devuelve un mensaje de error claro en lugar de permitir que el programa colapsé con una excepción. También comprendí la importancia de crear funciones puras y reutilizables para cada operación, lo que facilita las pruebas unitarias y el mantenimiento del código a largo plazo. Descubrí que, al separar la lógica de cálculo de la interfaz de usuario, el código se vuelve más modular y fácil de extender en el futuro, permitiendo agregar operaciones más complejas como potencias o raíces cuadradas sin afectar otras partes del sistema.

5. Bibliografía utilizada.

1. Python Software Foundation. (2024). Python 3.12 Documentation.
<https://docs.python.org/3/>
2. Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress.
<https://git-scm.com/book/en/v2>
3. Martin, R. C. (2009). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
4. Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

5. Atlassian. (2024). Feature Branch Workflow.

<https://www.atlassian.com/git/tutorials/comparing-workflows/feature-branch-workflow>

6. Goldberg, D. (1991). What Every Computer Scientist Should Know About Floating-Point Arithmetic. ACM Computing Surveys, 23(1), 5-48.

